23CSE211- DESIGN AND ANALYSIS OF ALGORTHIMS

NAME: VANGAPANDU KISHOR

ROLL NO: CH.SC.U4CSE24254

Huffman Coding

```c
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
struct Node {
    char data;
    unsigned freq;
    struct Node *left, *right;
};
struct Node* newNode(char data, unsigned freq) {
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->left = temp->right = NULL;
    temp->data = data;
    temp->freq = freq;
    return temp;
}
struct MinHeap {
    unsigned size;
    struct Node* arr[MAX];
};
void swap(struct Node** a, struct Node** b) {
    struct Node* t = *a;
    *a = *b;
    *b = t;
}
```

```c
void heapify(struct MinHeap* heap, int i) {
    int smallest = i;
    int left = 2*i + 1;
    int right = 2*i + 2;
    if (left < heap->size &&
        heap->arr[left]->freq < heap->arr[smallest]->freq)
        smallest = left;

    if (right < heap->size &&
        heap->arr[right]->freq < heap->arr[smallest]->freq)
        smallest = right;

    if (smallest != i) {
        swap(&heap->arr[i], &heap->arr[smallest]);
        heapify(heap, smallest);
    }
}
struct Node* extractMin(struct MinHeap* heap) {
    struct Node* temp = heap->arr[0];
    heap->arr[0] = heap->arr[--heap->size];
    heapify(heap, 0);
    return temp;
}
```

```c
void insertHeap(struct MinHeap* heap, struct Node* node) {
    int i = heap->size++;

    while (i && node->freq < heap->arr[(i-1)/2]->freq) {
        heap->arr[i] = heap->arr[(i-1)/2];
        i = (i-1)/2;
    }
    heap->arr[i] = node;
}S
struct Node* buildTree(char data[], int freq[], int size) {
    struct MinHeap heap;
    heap.size = 0;
    for (int i = 0; i < size; i++)
        heap.arr[heap.size++] = newNode(data[i], freq[i]);
    for (int i = (heap.size-1)/2; i >= 0; i--)
        heapify(&heap, i);
    while (heap.size > 1) {
        struct Node* left = extractMin(&heap);
        struct Node* right = extractMin(&heap);
        struct Node* top = newNode('$', left->freq + right->freq);
        top->left = left;
        top->right = right;
        insertHeap(&heap, top);
    }
    return extractMin(&heap);
}
```

```c
void printCodes(struct Node* root, int arr[], int top) {
    if (root->left) {
        arr[top] = 0;
        printCodes(root->left, arr, top + 1);
    }
    if (root->right) {
        arr[top] = 1;
        printCodes(root->right, arr, top + 1);
    }
    if (!root->left && !root->right) {
        printf("%c: ", root->data);
        for (int i = 0; i < top; i++)
            printf("%d", arr[i]);
        printf("\n");
    }
}
int main() {
    char data[] = {'A','B','C','D','E','F'};
    int freq[] = {5,9,12,13,16,45};
    int size = sizeof(data)/sizeof(data[0]);
    struct Node* root = buildTree(data, freq, size);
    int arr[MAX], top = 0;
    printf("Huffman Codes:\n");
    printCodes(root, arr, top);
    return 0;
}
```

Output:

```
vangapandu-kishor@vangapandu-kishor-IdeaPad-Slim-5-14IRH10:~/Downloads$ gcc -o huffman huffman.c
vangapandu-kishor@vangapandu-kishor-IdeaPad-Slim-5-14IRH10:~/Downloads$ ./huffman
Huffman Codes:
F: 0
C: 100
D: 101
A: 1100
B: 1101
E: 111
```

Time Complexity: O(N + M log M) :

The overall time complexity is **O(N + M log M)**, where **N** represents the length of the input data and **M** represents the number of distinct characters. Counting the frequency of characters requires **O(N)** time, while constructing the Huffman tree using a min-heap requires **O(M log M)** time due to repeated insertion and extraction operations.

Space Complexity: O(N + M) :

The space complexity is **O(N + M)**. Space is required to store the input data (**N**) and the Huffman tree nodes along with auxiliary data structures (**M**). Since the number of unique characters is limited (for example, 256 in ASCII), the space complexity is effectively linear in practice.

Job Sequencing Problem

```c
#include <stdio.h>
#include <stdlib.h>
typedef struct {
    int id;
    int deadline;
    int profit;
} Job;
int compare(const void *a, const void *b) {
    Job *j1 = (Job *)a;
    Job *j2 = (Job *)b;
    return j2->profit - j1->profit;
}
int main() {
    int n;
    printf("Enter number of jobs: ");
    scanf("%d", &n);
  Job jobs[n];

 for(int i = 0; i < n; i++) {
        printf("Enter deadline and profit for job %d: ", i + 1);
        scanf("%d %d", &jobs[i].deadline, &jobs[i].profit);
        jobs[i].id = i + 1;
    }
qsort(jobs, n, sizeof(Job), compare);
    int maxDeadline = 0;
```

```c
    for(int i = 0; i < n; i++)
        if(jobs[i].deadline > maxDeadline)
            maxDeadline = jobs[i].deadline;
    int slot[maxDeadline];
for(int i = 0; i < maxDeadline; i++)
        slot[i] = -1;
    int totalProfit = 0;
for(int i = 0; i < n; i++) {
     for(int j = jobs[i].deadline - 1; j >= 0; j--) {
          if(slot[j] == -1) {
                slot[j] = jobs[i].id;
                totalProfit += jobs[i].profit;
                break;
            }
        }
    }
 printf("\nScheduled Jobs: ");
for(int i = 0; i < maxDeadline; i++)
        if(slot[i] != -1)
            printf("J%d ", slot[i]);
 printf("\nTotal Profit = %d\n", totalProfit);

 return 0;
}
```

Output:

```
vangapandu-kishor@vangapandu-kishor-IdeaPad-Slim-5-14IRH10:~$ cd Downloads
vangapandu-kishor@vangapandu-kishor-IdeaPad-Slim-5-14IRH10:~/Downloads$ gcc -o jobsequence jobsequence.c
vangapandu-kishor@vangapandu-kishor-IdeaPad-Slim-5-14IRH10:~/Downloads$ ./jobsequence
Enter number of jobs: 14
Enter deadline and profit for job 1: 5 36
Enter deadline and profit for job 2: 8 21
Enter deadline and profit for job 3: 3 28
Enter deadline and profit for job 4: 4 19
Enter deadline and profit for job 5: 4 21
Enter deadline and profit for job 6: 9 13
Enter deadline and profit for job 7: 12 28
Enter deadline and profit for job 8: 14 25
Enter deadline and profit for job 9: 2 18
Enter deadline and profit for job 10: 7 20
Enter deadline and profit for job 11: 5 27
Enter deadline and profit for job 12: 1 22
Enter deadline and profit for job 13: 6 14
Enter deadline and profit for job 14: 3 26

Scheduled Jobs: J12 J14 J3 J11 J1 J13 J10 J2 J6 J7 J8
Total Profit = 260
```

Time Complexity: $O(N^2)$:

The program takes $O(N^2)$ time because it uses bubble sort to order jobs by profit and a nested loop to assign jobs to available time slots in the worst case.

Space Complexity: O(N) :

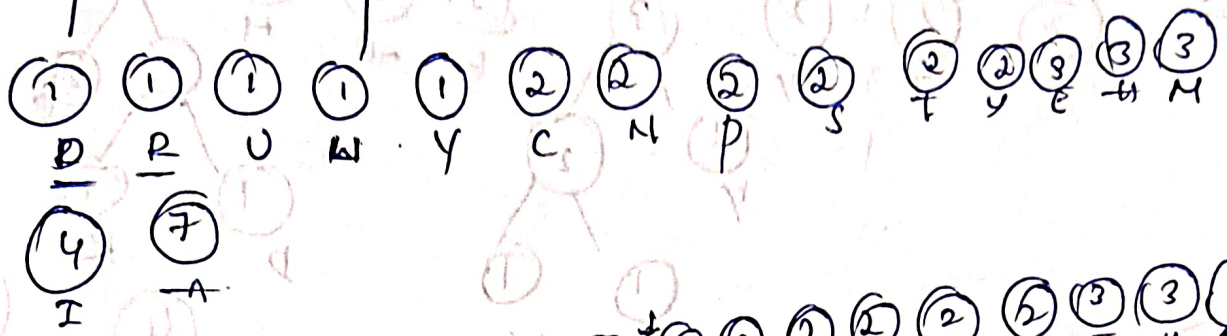
The program requires O(N) space to store the jobs and the time slot array, which are proportional to the number of jobs.

Q Using Huffman Coding write min encoding bits for phrase

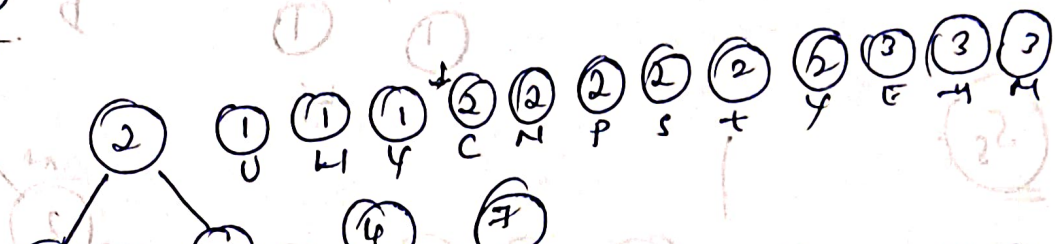
AMRITA VISHWA VIDYAPEETHAM CHENNAI CAMPUS

A	M	R	I	T	V	S	H	W	D	Y	P	E	C	N	U
7	3	1	4	2	2	2	3	1	1	1	2	3	2	2	1

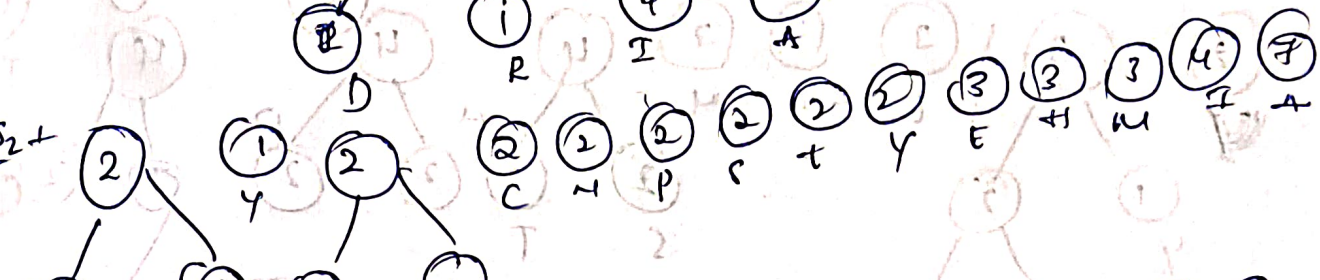
Arrange in ascending Order



Step 1 :

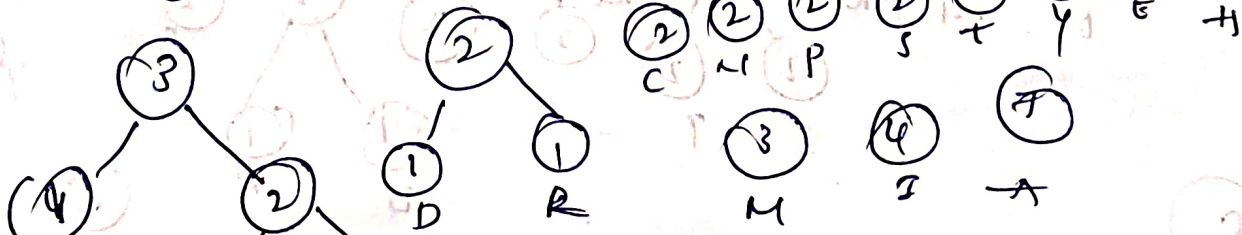


S2 :

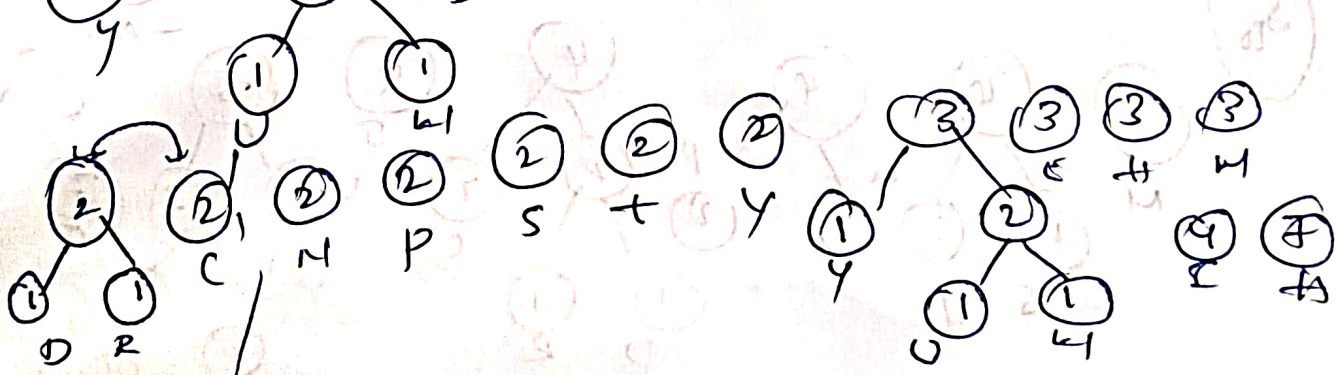


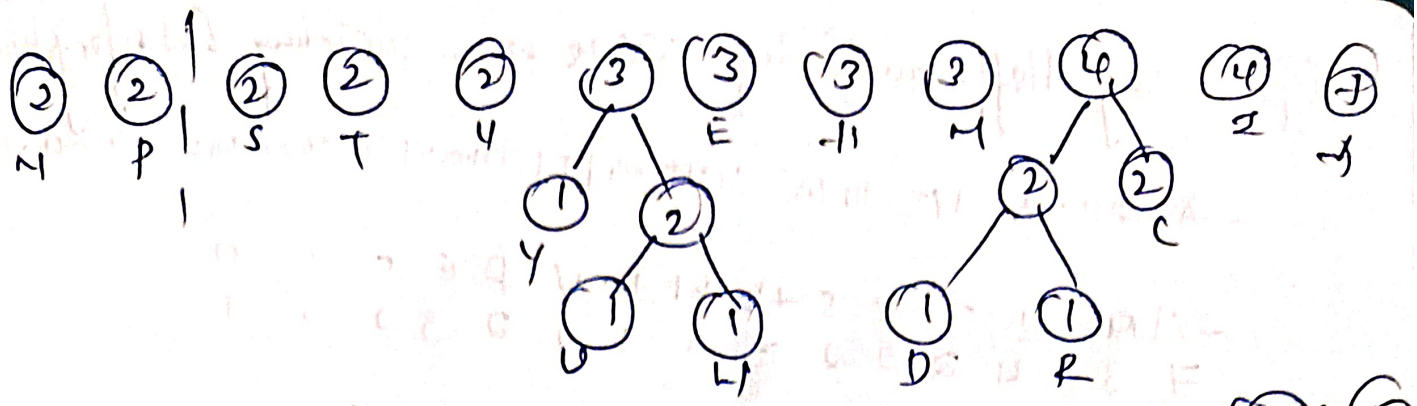
S3 :

S4 :

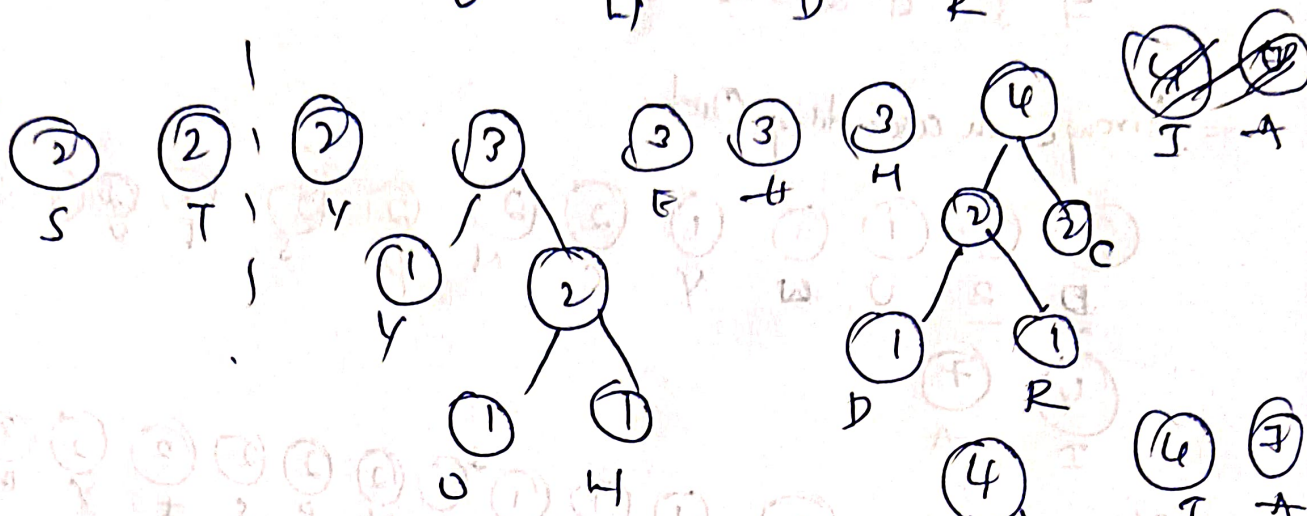


S5 :

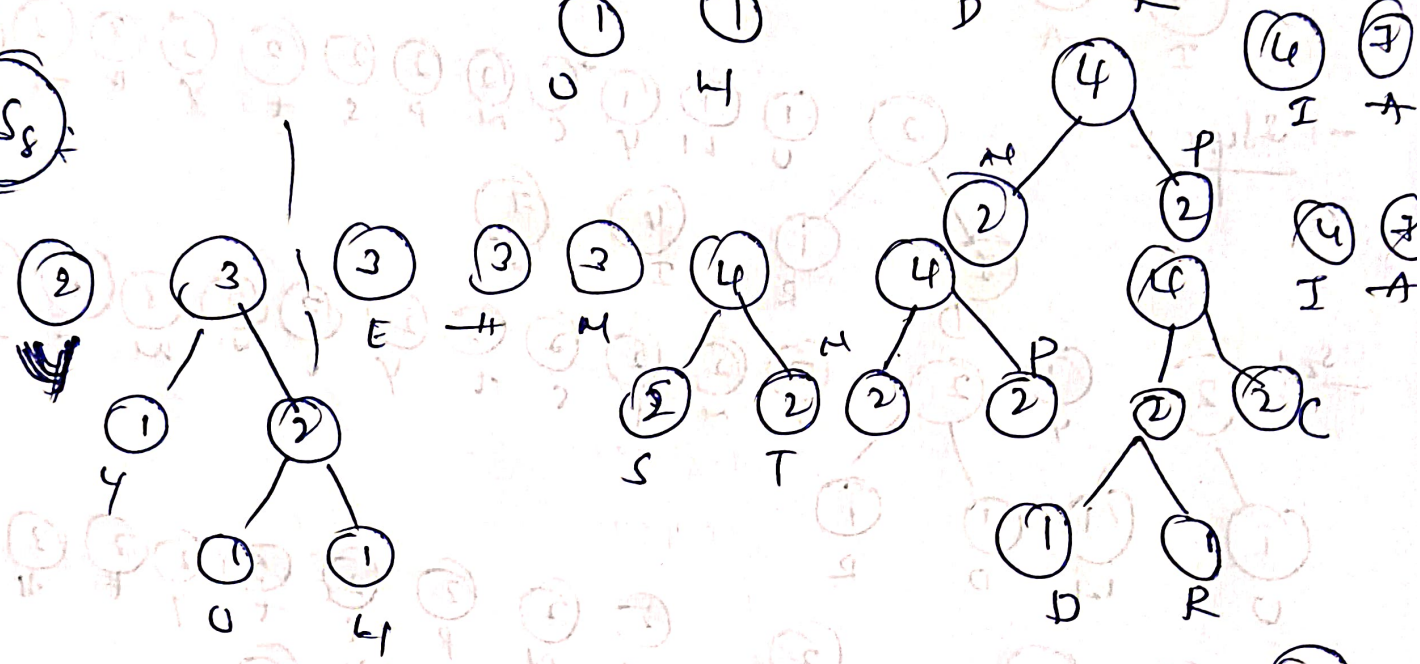




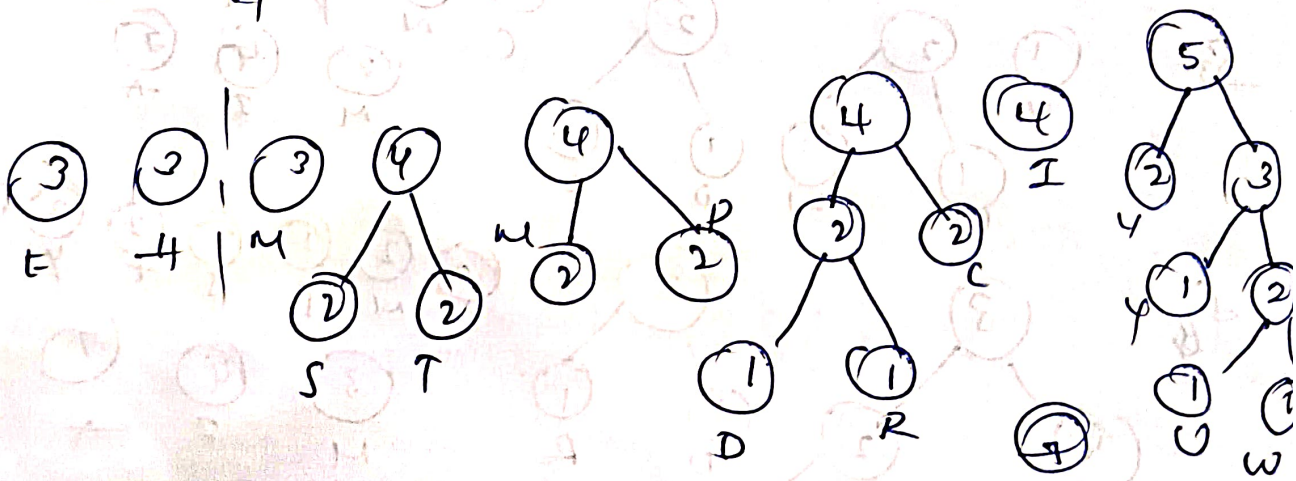
S_7:



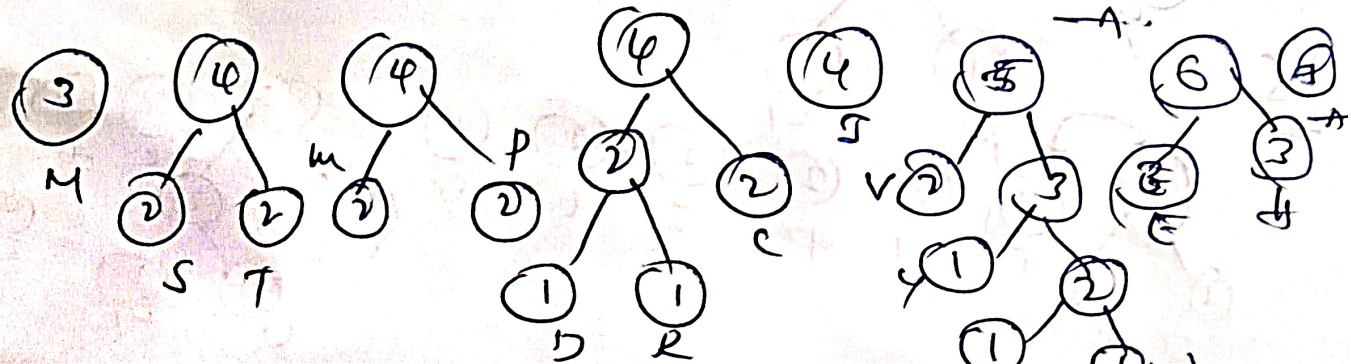
S_8:



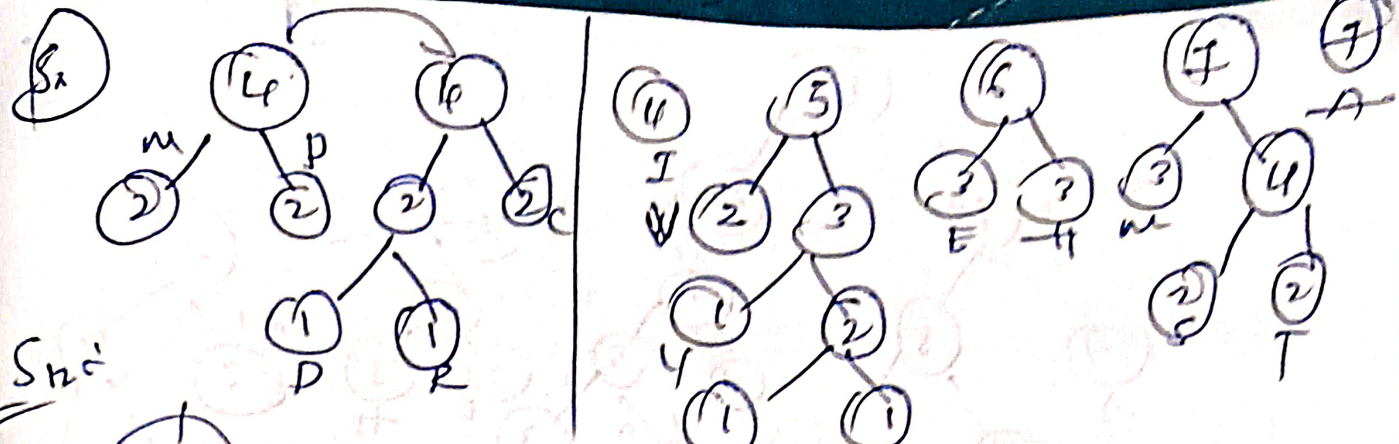
S_9:



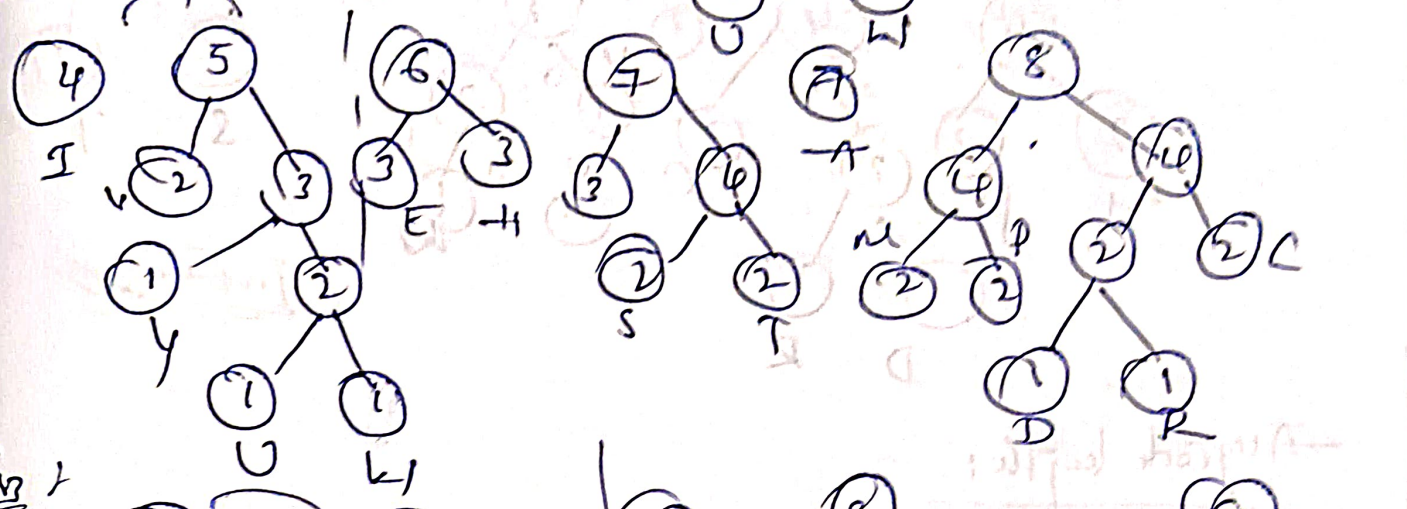
S_{10}



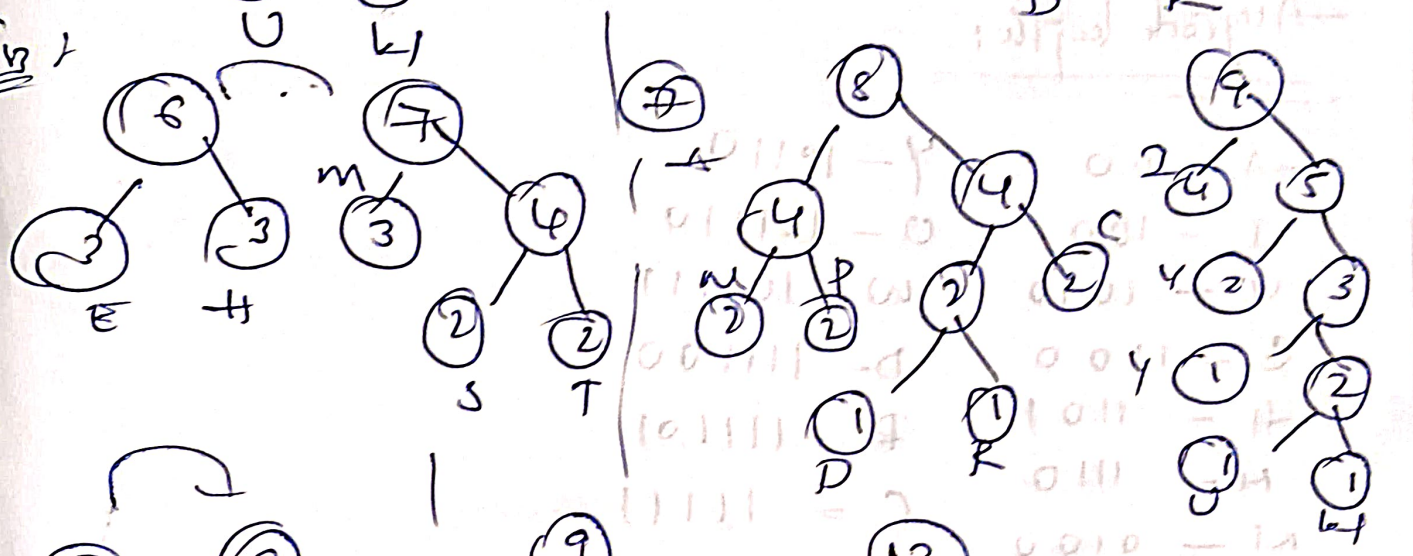
Sn



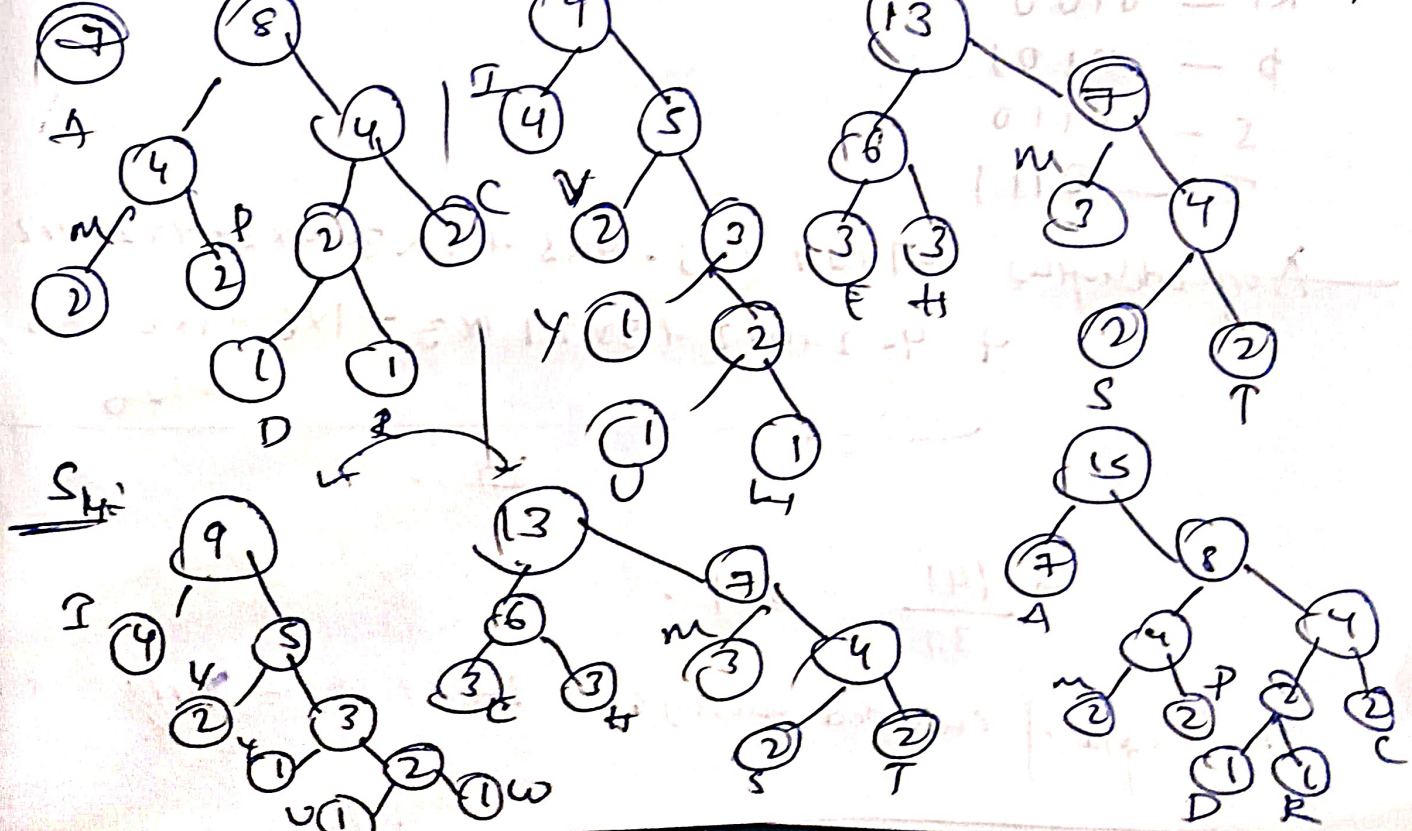
Sn-1

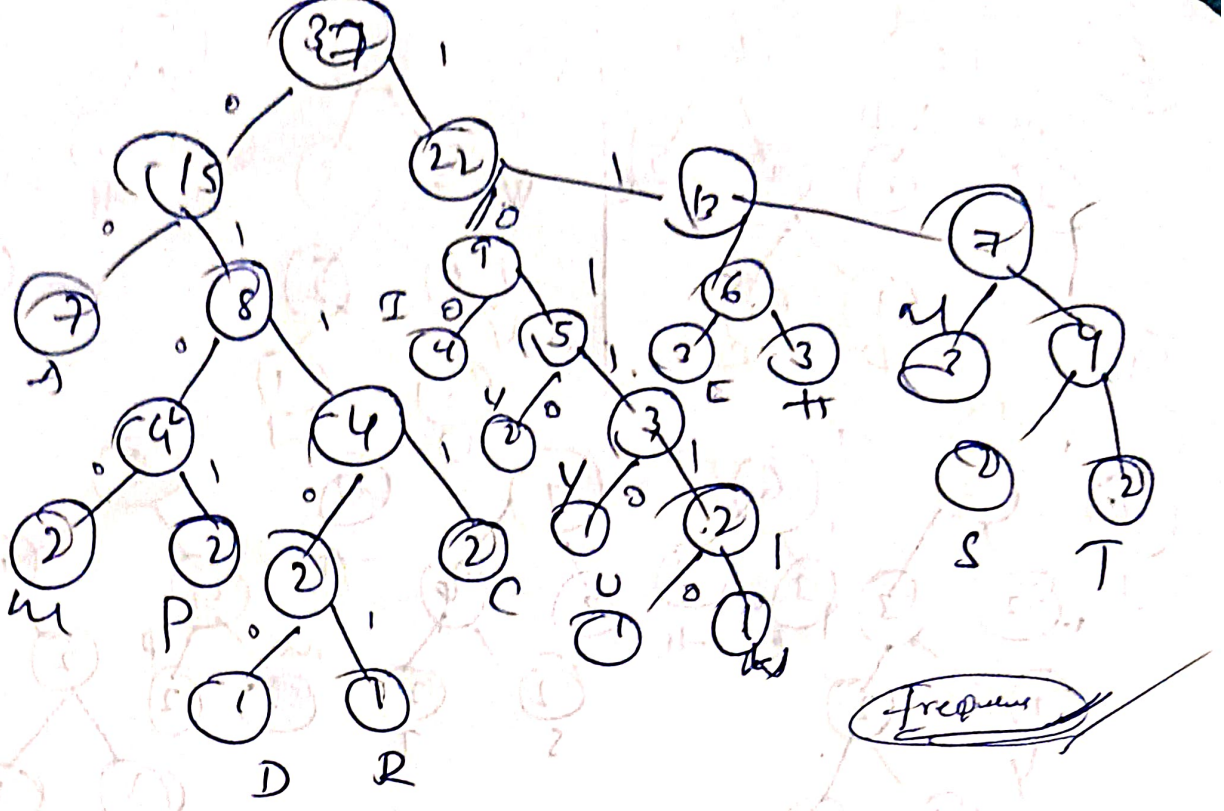


Sn-2



Sn-3





Avg code length:

A – 00
I – 100
U – 1010
E – 1100
H – 1101
M – 1110
N – 0100
P – 0101
S – 0110
T – 0111

Y – 10110
O – 101110
W – 101111
D – 111100
R – 111101
C – 11111

Avg Code length = $\frac{7\times2 + 4\times3 + 4\times3 + 4\times3 + 4\times3 + 4\times2 + 4\times2 + 4\times2 + 4\times2 + 5\times2 + 1\times5 + 1\times6 + 1\times6 + 1\times6 + 1\times6}{37}$

$= \frac{141}{37} = 3.81$.

Length of encoded message $= 37\times3.81 = 141$

(7) Job scheduling :-

Let there be 14 jobs for sequencing whose profits are

$P = \{36, 21, 28, 19, 21, 13, 28, 25, 18, 20, 27, 22, 14, 26$

Times are 5, 8, 3, 4, 4, 9, 12, 14, 2, 7, 5, 1, 6, 3

J_1	J_2	J_3	J_4	J_5	J_6	J_7	J_8	J_9	J_{10}	J_{11}	J_{12}	J_{13}	J_{14}
36	21	28	19	21	13	28	25	18	20	27	22	14	26
5	8	3	4	4	9	12	14	2	7	5	1	6	3

J_2	J_9	J_3	J_5	J_1	J_{13}	J_{10}	J_2	J_6		J_4	J_7		J_8

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14